

High-Quality Sender Diagnostics with Constexpr Exceptions

Document #: P3557R2
Date: 2025-05-16
Project: Programming Language C++
Audience: LWG Library Working Group
Reply-to: Eric Niebler
[<eric.niebler@gmail.com>](mailto:eric.niebler@gmail.com)

Contents

- [1 Introduction](#)
- [2 Executive Summary](#)
- [3 Revision History](#)
 - [3.1 R2](#)
 - [3.2 R1](#)
 - [3.3 R0](#)
- [4 Motivation](#)
 - [4.1 Example](#)
 - [4.1.1 With constexpr exceptions:](#)
 - [4.1.2 Without constexpr exceptions:](#)
- [5 Proposed Design](#)
 - [5.1 `get\_completion\_signatures`](#)
 - [5.1.1 *Non*-non-dependent senders](#)
 - [5.1.2 Implementation](#)
 - [5.2 `sender\_in`](#)
 - [5.3 *basic-sender*](#)
 - [5.4 `dependent\_sender`](#)
- [6 Implementation Experience](#)
- [7 Proposed Wording](#)
- [8 Acknowledgements](#)
- [9 References](#)

§ 1 Introduction

The hardest part of writing a sender algorithm is often the computation of its completion signatures, an intricate meta-programming task. Using sender algorithms incorrectly leads to large, incomprehensible errors deep within the completion-signatures meta-program. What is needed is a way to propagate type errors automatically to the API boundary where they can be reported concisely, much the way exceptions do for runtime errors.

Support for exceptions during constant-evaluation was recently accepted into the Working Draft for C++26. We can take advantage of this powerful new feature to easily propagate type errors during the computation of a sender’s completion signatures. This significantly improves the diagnostics users are likely to encounter while also simplifying the job of writing new sender algorithms.

§ 2 Executive Summary

This paper proposes the following changes to the working draft with the addition of [P3164R4]. Subsequent sections will address the motivation and the designs in detail.

1. Change `std::execution::get_completion_signatures` from a customization point object that accepts a sender and (optionally) an environment to a consteval function template that takes no arguments, as follows:

Before	After
<pre>inline constexpr struct get_completion_signatures_t { template <class Sndr, class... Env> auto operator()(Sndr&&, Env&&...) const -> see below; } get_completion_signatures {};</pre>	<pre>template <class Sndr, class... Env> constexpr auto get_completion_signatures() -> valid-completion-signatures auto;</pre>

2. Change the mechanism by which senders customize `get_completion_signatures` from a member function that accepts the cv-qualified sender object and an optional environment object to a static constexpr function template that take the sender and environment types as template parameters.

Before	After
<pre>struct my_sender { template <class Self, class... Env> requires some-predicate<Self, Env...> auto get_completion_signatures(this Self&&, Env&&) {</pre>	<pre>struct my_sender { template <class Self, class... Env> static constexpr auto get_completion_signatures() {</pre>

Before	After
<pre> return completion_signatures</* ... */>(); } ... }; </pre>	<pre> if constexpr (!some- predicate<Self, Env...>) { throw a-helpful-diagnostic(); // <--- LOOK! } return completion_signatures</* ... */>(); } ... }; </pre>

3. Change the `sender_in<Sender, Env...>` concept to test that `get_completion_signatures<Sndr, Env...>()` is a constant expression.

Before	After
<pre> template<class Sndr, class... Env> concept sender_in = sender<Sndr> && (queryable<Env> &&...) && requires (Sndr&& sndr, Env&&... env) { { get_completion_signatures(std::forward<Sndr> (sndr), std::forward<Env> (env)...) } -> valid-completion- signatures; }; </pre>	<pre> template <auto> concept is-constant = true; // exposition only template<class Sndr, class... Env> concept sender_in = sender<Sndr> && (queryable<Env> &&...) && is-constant<get_completion_signatures<Sndr, Env...>()>; </pre>

4. In the exposition-only *basic-sender* class template, specify under what conditions its `get_completion_signatures` static member function is ill-formed when called without an `Env` template parameter (see proposed wording for details).
5. Add a `dependent_sender` concept that is modeled by sender types that do not know how they will complete independent of their execution environment.
6. Remove the `transform_completion_signatures` alias template (to be later replaced with a `consteval` function that does the same job and that throws on error).

§ 3 Revision History

§ 3.1 R2

- Move the nice-to-haves into a separate paper (TBD).
- Drop support for specifying a sender’s completion signatures with a nested type alias.

- Promote the exposition-only *dependent-sender-error* class to `std::execution::dependent_sender_error`.
- Change *basic-sender::check-types* and all its customizations from `constexpr` to `constexpr`.
- Merge the proposed wording of [P3164R4] into this paper’s proposed wording for ease of wording review.

§ 3.2 R1

Since R0, a significant fraction of C++26’s `std::execution` has been implemented with the design changes proposed by this paper. Several bugs in R0 have been found and fixed as a result.

In addition, this paper exposed [several bugs](#) in the Working Draft for C++26. As those bugs relate to the computation of completion signatures, R1 integrates the proposed fixes for those bugs.

- Fix the specification of `std::execution::get_completion_signatures` which was the victim of an incomplete last-minute edit.
- Add support for awaitables, which was dropped accidentally, back to `get_completion_signatures`.
- Change proposed replacement for `transform_completion_signatures` to take a transform function for the stopped completion signature instead of a `completion_signatures` object.
- Add missing semantic constraints to *impls-for<Tag>::check-types* customizations.
- Add missing syntactic constraint on *basic-sender<Tag>::get_completion_signatures*.
- Strengthen the *Mandates* on `std::execution::connect`.
- Fixed the constraint on `stopped_as_optional.transform_sender(sndr, env)`.
- Report type errors from the `let_*`, `schedule_from`, `into_variant`, `split`, and `when_all` algorithms when the predecessor’s result datums cannot be copied into intermediate storage.
- Add proposed wording for the nice-to-have features:
 - `eptr_completion_if`
 - `make_completion_signatures`
 - `transform_completion_signatures` (a `constexpr` function version)
 - `invalid_completion_signature`
- Integrate proposed fix for [cplusplus/sender-receiver#311](#), “`stopped_as_optional` tests constraints satisfaction of self instead of child”.
- Integrate proposed fix for [cplusplus/sender-receiver#317](#), “Several algorithms are not applying *FWD-ENV* as they should”.

- Integrate proposed fix for [cplusplus/sender-receiver#318](#), “specification of `as-sndr2(Sig)` in `[exec.let]` is incomplete”.
- Integrate proposed fix for [cplusplus/sender-receiver#319](#), “`let_[*].transform_env` is specified in terms of the `let_*` sender itself instead of its child”.
- Fix paragraph numbering.

§ 3.3 R0

- Initial revision

§ 4 Motivation

This paper exists principally to improve the experience of users who make type errors in their sender expressions by leveraging exceptions during constant- evaluation. It is a follow-on of [\[P3164R4\]](#), which defines a category of “non-dependent” senders that can and must be type-checked early.

Senders have a construction phase and a subsequent connection phase. Prior to P3164, all type-checking of senders happened at the connection phase (when a sender is connected to a receiver). P3164 mandates that the sender algorithms type-check non-dependent senders, moving the diagnostic closer to the source of the error.

This paper addresses the *quality* of those diagnostics, as well as the diagnostics users encounter when a dependent sender fails type-checking at connection time.

Senders are expression trees, and type errors can happen deep within their structure. If programmed naively, ill-formed senders would generate megabytes of incomprehensible diagnostics. The challenge is to report type errors *concisely* and *comprehensibly*, at the right level of abstraction.

Doing this requires propagating domain-specific descriptions of type errors out of the completion signatures meta-program so they can be reported concisely. Such error detection and propagation is very cumbersome in template meta-programming.

The C++ solution to error propagation is exceptions. With the adoption of [\[P3068R6\]](#), C++26 has gained the ability to throw and catch exceptions during constant-evaluation. If we express the computation of completion signatures as a `constexpr` meta-program, we can use exceptions to propagate type errors. This greatly improves diagnostics and even simplifies the code that computes completion signatures.

This paper proposes changes to `std::execution` that make the computation of a sender’s completion signatures an evaluation of a `constexpr` function. It also specifies the conditions under which the computation is to exit with an exception.

§ 4.1 Example

How good are `std::execution`’s diagnostics with `constexpr` exceptions? Let’s consider a user that makes a simple mistake in their sender expression:

```
using stdex = std::execution;
auto sndr = stdex::just(42) | stdex::then([]() { /*...*/ });
```

This is an error because the nullary lambda cannot be called with the integer that `just(42)` completes with.

With `constexpr` exceptions, the resulting diagnostic is blissfully direct:

§ 4.1.1 With `constexpr` exceptions:

```
<source>:658:3: error: call to immediate function 'operator|
<_basic_sender<just_t, _tupl<nullptr, int>>>'
is not a constant expression
2212 | auto sndr = just(42) | then([](){});
      | ^
<source>:658:3: note: 'operator|<_basic_sender<just_t,
_tupl<nullptr, int>>>' is an immediate function
because its body contains a call to an immediate function
'_make_sender<then_t, (lambda at <source>:2212:
31), _basic_sender<just_t, _tupl<nullptr, int>>>' and that
call is not a constant expression
1912 | return transform_sender(dom,
_make_sender(Algorithm, std::move(_self.data_), sndr));
      |
^~~~~~
<source>:658:3: note: unhandled exception of type
'_sender_type_check_failure<const char *, IN_ALGORITHM<
then>, WITH_FUNCTION ((lambda at <source>:2212:31)),
WITH_ARGUMENTS (int)>' with content {{{}}, &"The fun
ction passed to std::execution::then is not callable with
the values sent by the predecessor sender."[0]}
thrown from here
876 | throw _sender_type_check_failure<Values...[0],
What...>(values...);
      | ^
```

The above is the *complete* diagnostic, regardless of how deeply nested the type error is.

In contrast, the following is the diagnostic we get without `constexpr` exceptions. Notice that the backtrace is truncated; the actual error is considerably longer.

§ 4.1.2 Without `constexpr` exceptions:

```
<source>:1912:36: error: call to immediate function 'operator|
<_basic_sender<just_t, _tupl<nullptr, int>>>' is not a constant
expression
2212 | auto sndr = just(42) | then([](){});
      | ^
<source>:1912:36: note: 'operator|<_basic_sender<just_t, _tupl<nullptr, int>>>'
is an immediate function because its body contains a call to an immediate function
'_make_sender<then_t, (lambda at <source>:2212:31), _basic_sender<just_t, _tupl<nullptr,
int>>>' and that call is not a constant expression
1912 | return transform_sender(dom, _make_sender(Algorithm,
std::move(_self.data_), sndr));
      |
^~~~~~
<source>:876:5: note: subexpression not valid in a constant expression
876 | throw _sender_type_check_failure<Values...[0], What...>
```

```

(values...);
|
<source>:1990:16: note: in call to
'invalid_completion_signature<IN_ALGORITHM<then>, WITH_FUNCTION ((lambda at
<source>:2212:3
1)), WITH_ARGUMENTS (int), const char *(&"The function passed to
std::execution::then is not callable with the values sent by
the predecessor sender."[0 ]]'
1990 |         return invalid_completion_signature<IN_ALGORITHM<then>,
|         ^~~~~~
1991 |                                     struct WITH_FUNCTION(Fn),
|                                     ~~~~~~
1992 |                                     struct
WITH_ARGUMENTS(As...)>(|
|
|
1993 |         "The function passed to std::execution::then is not callable
with the"
|
|
1994 |         " values sent by the predecessor sender.");
|         ^~~~~~
<source>:885:10: note: in call to 'fn.operator()<int>()'
885 |         return fn.template operator()<As...>();
|         ^~~~~~
<source>:897:12: note: in call to '_transform_expr<int, _impls_for<then_t>::
(lambda at <source>:1988:5)>(fn..)'
897 |         return ::_transform_expr<As...>(fn);
|         ^~~~~~
<source>:923:14: note: in call to '_apply_transform<int, _impls_for<then_t>::
(lambda at <source>:1988:5)>(fn..)'
923 |         return _apply_transform<Ts...>(value_fn);
|         ^~~~~~
<source>:930:13: note: in call to 'transform1.operator()<set_value_t, int>
(nullptr)'
930 |         return (transform1(sigs) +...+ completion_signatures());
|         ^~~~~~
<source>:804:12: note: (skipping 6 calls in backtrace; use -fconstexpr-
backtrace-limit=0 to see all)
804 |         return fn(_normalized_sig_t<Sigs>()...);
|         ^
<source>:1206:31: note: in call to
'get_completion_signatures<_basic_sender<then_t, (lambda at <source>:2212:31),
_basic_sende
r<just_t, _tupl<nullptr, int>>>>()'
1206 |         return _CHECKED_COMPLSIGS(_GET_COMPLSIGS(Sndr, Env...));
|         ^~~~~~
<source>:1188:3: note: expanded from macro 'GET_COMPLSIGS'
1188 |         std::remove_reference_t<Sndr>::template get_completion_signatures<Sndr
__VA_OPT__(,) __VA_ARGS__>()
|         ^
<source>:1190:34: note: expanded from macro '_CHECKED_COMPLSIGS'
1190 |         #define _CHECKED_COMPLSIGS(...) (__VA_ARGS__,
::_checked_complsig<decltype(__VA_ARGS__)>())
|         ^~~~~~
<source>:1230:10: note: in call to
'_get_completion_signatures_helper<_basic_sender<then_t, (lambda at
<source>:2212:31), _bas
ic_sender<just_t, _tupl<nullptr, int>>>>()'
1230 |         return _get_completion_signatures_helper<Sndr>();
|         ^~~~~~
<source>:1872:12: note: in call to
'get_completion_signatures<_basic_sender<then_t, (lambda at <source>:2212:31),
_basic_sende
r<just_t, _tupl<nullptr, int>>>>()'
1872 |         (void) get_completion_signatures<Sender>();
|         ^~~~~~
<source>:1912:36: note: in call to '_make_sender<then_t, (lambda at

```

```

<source>:2212:31), _basic_sender<just_t, _tuple<nullptr, in
t>>>({{}}), {}, {{}}, {{}}, {{42      }}})'
1912 |         return transform_sender(dom, _make_sender(Algorithm,
std::move(_self.data_), sndr));
|
^~~~~~
<source>:2212:15: note: in call to 'operator|<_basic_sender<just_t,
_tuple<nullptr, int>>>({{}}), {}, {{42      }}), {{}})'
2212 |         auto sndr = just(42      ) | then([](){});
|         ^~~~~~

```

The code that generated these diagnostics can be found [here](https://godbolt.org/z/rPEqWz693) or at <https://godbolt.org/z/rPEqWz693>

§ 5 Proposed Design

§ 5.1 get_completion_signatures

In the Working Draft, a sender's completion signatures are determined by the type of the expression `std::execution::get_completion_signatures(sndr, env)` (or, after P3164, `std::execution::get_completion_signatures(sndr)` for non-dependent senders). Only the type of the expression matters; the expression itself is never evaluated.

In the design proposed by this paper, the `get_completion_signatures` expression must be constant-evaluated in order use exceptions to report errors. To make it ammenable to constant evaluation, it must not accept arguments with runtime values, so the expression is changed to `std::execution::get_completion_signatures<Sndr, Env...>()`, where `get_completion_signatures` is a consteval function.

If an unhandled exception propagates out of `get_completion_signatures` the program is ill-formed (because `get_completion_signatures` is consteval). The diagnostic displays the type and value of the exception.

`std::execution::get_completion_signatures<Sndr, Env...>()` in turn calls `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`, which computes the completion signatures or throws as appropriate, as shown below:

```

namespace exec = std::execution;

struct void_sender {
    using sender_concept = exec::sender_t;

    template <class Self, class... Env>
    static constexpr auto get_completion_signatures() {
        return exec::completion_signatures<exec::set_value_t>(>());
    }

    /* ... more ... */
};

```

To better support the constexpr value-oriented programming style, calls to `std::execution::get_completion_signatures` from a constexpr function are never ill-formed, and their return type is always a specialization of the `completion_signatures`

template. `std::execution::get_completion_signatures` reports errors by failing to be a constant expression.

§ 5.1.1 *Non-non-dependent senders*

[P3164R4] introduces the concept of non-dependent senders: senders that have the same completion signatures regardless of the receiver's execution environment. For a sender type `DependentSndr` whose completions *do* depend on the environment, what should happen when the sender's completions are queried without an environment? That is, what should the semantics be for `get_completion_signatures<DependentSndr>()`?

`get_completion_signatures<DependentSndr>()` should follow the general rule: it should be well-formed in a `constexpr` function, and it should have a `completion_signatures` type. That way, sender adaptors do not need to do anything special when computing the completions of child senders that are dependent. So `get_completion_signatures<DependentSndr>()` should throw.

If `get_completion_signatures<Sndr>()` throws for dependent senders, and it also throws for non-dependent senders that fail to type-check, how then do we distinguish between valid dependent and invalid non-dependent senders? We can distinguish by checking the type of the exception.

An example will help. Consider the `read_env(q)` sender, a dependent sender that sends the result of calling `q` with the receiver's environment. It cannot compute its completion signatures without an environment. The natural way for the `read_env` sender to express that is to require an `Env` parameter to its customization of `get_completion_signatures`:

```
namespace exec = std::execution;

template <class Query>
struct read_env_sender {
    using sender_concept = exec::sender_t;

    template <class Self, class Env> // NOTE: Env is not optional!
    static constexpr auto get_completion_signatures() {
        if constexpr (!std::invocable<Query, Env>) {
            throw exception-type-goes-here();
        } else {
            using Result = std::invoke_result_t<Query, Env>;
            return exec::completion_signatures<exec::set_value_t(Result)>
                ();
        }
    }
};

/* ... more ... */
```

That makes `read_env_sender<Q>::get_completion_signatures<Sndr>()` an ill-formed expression, which the `get_completion_signatures` function can detect. In such cases, it would throw an exception of a special type that it can catch later when distinguishing between dependent and non-dependent senders.

§ 5.1.2 *Implementation*

Since the design has several parts, reading the implementation of `get_completion_signatures` is probably the easiest way to understand it. The implementation is shown below with comments describing the parts.

```
// This macro expands to an invocation of SNDR's
// get_completion_signatures
// customization.
#define GET_COMPLSIGS(SNDR, ...)
    std::remove_reference_t<SNDR>::template \
    get_completion_signatures<SNDR __VA_OPT__(,) __VA_ARGS__>()

// This macro expands to an evaluation of EXPR, followed by an
// invocation
// of the _checked_complsig function which validates its type.
#define CHECK_COMPLSIGS(EXPR) (EXPR, \
    _check_complsig<decltype(EXPR)>())

// This helper ensures that the passed type is indeed a
// specialization of the
// completion_signatures class template, and throws if it is not.
template <class Completions>
constexpr auto _check_complsig() {
    if constexpr (valid-completion-signatures<Completions>)
        // We got a type that is a specialization of the
        // completion_signatures
        // class template representing the sender's completions. Return
        // it.
        return Completions();
    else
        // throw unconditionally but make sure the return type is
        // deduced to
        // be `completion_signatures<>`. That way, a sender adaptor can
        // ask
        // a child for its completions and take for granted that the
        // return
        // type is a specialization of `completion_signatures<>`
        return (throw unspecified, completion_signatures());
}

template <class Sndr, class... Env>
constexpr auto _get_completion_signatures_helper() {
    // The following `if` tests whether GET_COMPLSIGS(Sndr, Env...)
    // is a well-formed expression.
    if constexpr (requires { GET_COMPLSIGS(Sndr, Env...); }) {
        // The GET_COMPLSIGS(Sndr, Env...) expression is well-formed,
        // but it may
        // throw an exception or otherwise fail to be a constant
        // expression.
        // By evaluating it, we cause its exception and its non-
        // constexpr-ness to
        // propagate. Then CHECK_COMPLSIGS ensures that its type is
        // indeed
        // a specialization of the completion_signatures class template.
        return CHECK_COMPLSIGS(GET_COMPLSIGS(Sndr, Env...));
    }
    // The following `if` does the same as above, but for
    // GET_COMPLSIGS(Sndr).
    // A non-dependent sender may announce itself by way of this
    // signature.
}
```

```

else if constexpr (requires { GET_COMPLSIGs(Sndr); }) {
    // Same as above: propagate any exceptions and non-constexpr-
    // ness, and
    // verify the expression has the right type.
    return CHECK_COMPLSIGs(GET_COMPLSIGs(Sndr));
}
// We can't recognize Sndr as a sender proper, but maybe it is
// awaitable,
// in which case we can adapt it to be a sender.
else if constexpr (is-awaitable<Sndr, env-promise<Env>...>) {
    return completion_signatures<
        SET-VALUE-SIG(await-result-type<Sndr, env-promise<Env>...>),
        // ([exec.snd.concepts])
        set_error_t(exception_ptr),
        set_stopped_t()>();
}
// If none of the above expressions are well-formed, then we don't
// know
// the sender's completions. If we are testing without an
// environment, then
// we assume Sndr is a dependent sender. Throw an exception that
// communicates that.
else if constexpr (sizeof...(Env) == 0) {
    return (throw dependent_sender_error(),
        completion_signatures());
}
else {
    // We cannot compute the completion signatures for this sender
    // and
    // environment. Give up and throw an exception.
    return (throw unspecified, completion_signatures());
}
}

template <class Sndr>
constexpr auto get_completion_signatures() ->
    _valid_completion_signatures auto {
    // There is no environment, which means we are asking for the
    // sender's non-
    // dependent completion signatures. If the sender is dependent,
    // this will
    // exit with a special exception type.
    return _get_completion_signatures_helper<Sndr>();
}

template <class Sndr, class Env>
constexpr auto get_completion_signatures() ->
    _valid_completion_signatures auto {
    // Apply a lazy sender transform if one exists before computing
    // the completion signatures:
    using Domain = decltype(_get_domain_late(std::declval<Sndr>(),
        std::declval<Env>()));
    using NewSndr = decltype(transform_sender(Domain(),
        std::declval<Sndr>(), std::declval<Env>()));

    return _get_completion_signatures_helper<NewSndr, Env>();
}

```

Given this definition of `get_completion_signatures`, we can implement a `dependent_sender` concept as follows:

```
// Returns true when get_completion_signatures<Sndr>() throws a
// dependent_sender_error or a type derived from it. Returns false
// when
// get_completion_signatures<Sndr>() returns normally (Sndr is non-
// dependent).
template <class Sndr>
constexpr bool is-dependent-sender-helper() try {
    (void) get_completion_signatures<Sndr>();
    return false;
} catch (dependent_sender_error&) {
    return true;
}

// If get_completion_signatures<Sndr>() throws an exception other
// than
// dependent_sender_error, then is-dependent-sender-helper<Sndr>()
// will
// fail to be a constant expression and so will not be a valid non-
// type
// template parameter to bool_constant. Therefore,
// dependent_sender<Sndr> will
// be false.
template <class Sndr>
concept dependent_sender =
    sender<Sndr> &&
    bool_constant<is-dependent-sender-helper<Sndr>()>::value;
```

After the adoption of [P3164R4], the sender algorithms are all required to return senders that are either dependent or else that type-check successfully. This paper proposes adding that type-checking as a *Mandates* on the exposition-only *make_sender* function template that all the algorithms use to construct their return value.

Users who define their own sender algorithms can use `dependent_sender` and `get_completion_signatures` to perform early type-checking of their own sender types using a helper such as the following:

```
template <class Sndr>
constexpr auto _type_check_sender(Sndr sndr) {
    if constexpr (!dependent_sender<Sndr>) {
        // This line will fail to compile if Sndr fails its type
        // checking. We
        // don't want to perform this type checking when Sndr is
        // dependent, though.
        // Without an environment, the sender doesn't know its
        // completions.
        (void) get_completion_signatures<Sndr>();
    }
    return sndr;
}
```

Using this helper, a then algorithm might type-check its returned senders as follows:

```
inline constexpr struct then_t {
    template <sender Sndr, class Fn>
```

```

auto operator()(Sndr sndr, Fn fn) const {
    return _type_check_sender(_then_sender{std::move(sndr),
        std::move(fn)});
}
} then {};

```

§ 5.2 sender_in

With the above changes, we need to tweak the `sender_in` concept to require that `get_completion_signatures<Sndr, Env...>()` is a constant expression.

The changes to `sender_in` relative to [P3164R4] are as follows:

```

template <auto>
    concept is-constant = true; // exposition only

template<class Sndr, class... Env>
    concept sender_in =
        sender<Sndr> &&
        (sizeof...(Env) <= 1)
        (queryable<Env> &&...) &&
        is-constant<get_completion_signatures<Sndr, Env...>()>;
requires (Sndr&& sndr, Env&&... env) {
    {get_completion_signatures(std::forward<Sndr>(sndr),
        std::forward<Env>(env)...) }
}-> valid-completion-signatures;
};

```

§ 5.3 basic-sender

The sender algorithms are expressed in terms of the exposition-only class template *basic-sender*. The mechanics of computing completion signatures is not specified, however, so very little change there is needed to implement this proposal.

We do, however, have to say when `basic-sender::get_completion_signatures<Sndr>()` is ill-formed. In [P3164R4], non-dependent senders are dealt with by discussing whether or not a sender's potentially-evaluated completion operations are dependent on the type of the receiver's environment. In this paper, we make a similar appeal when specifying whether or not `basic-sender::get_completion_signatures<Sndr>()` is well-formed.

§ 5.4 dependent_sender

Users who write their own sender adaptors will also want to perform early type-checking of senders that are not dependent. Therefore, they need a way to determine whether or not a sender is dependent.

In the section `get_completion_signatures` we show how the concept `dependent_sender` can be implemented in terms of this paper's `get_completion_signatures` function template. By making this a public-facing concept, we give sender adaptor authors a way to do early type-checking, just like the standard adaptors.

§ 6 Implementation Experience

A significant fraction of `std::execution` has been implemented with this design change. It can be found on [Compiler Explorer¹](#) and in [this GitHub gist²](#). This implementation includes all the design elements that would stress the completion signature computation including:

- Both dependent and non-dependent senders and sender adaptors, with tests for both early and late diagnosis of type errors.
- The exposition-only *basic-sender* has been implemented with the changes `get_completion_signatures` customization syntax and the new *impls-for<Tag>::check-types* customization, which has been used to implement and type-check all the sender types, as specified.
- A sender that is implemented via a lowering to another sender type via a lazy `transform_sender` customization (`stopped_as_optional`).
- Using awaitables as senders and *vice versa*.

§ 7 Proposed Wording

[Editor's note: This wording is relative to the current working draft with the addition of [P3164R4]]

[Editor's note: Change [async.ops]/13 as follows:]

- ¹³ A *completion signature* is a function type that describes a completion operation. An asynchronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates ([basic.def.odr]). For a completion function set, receiver `rcvr`, and pack of arguments `args`, let `c` be the completion operation `set(rcvr, args...)`, and let `F` be the function type `decltype(auto(set))` (`decltype(args)...`). A completion signature `Sig` is associated with `c` if and only if *MATCHING-SIG*(`Sig`, `F`) is true ([exec.general]). Together, a sender type and an environment type `Env` determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type `Env`. The type of the receiver does not affect an asynchronous operation's completion signatures, only the type of the receiver's environment. A non-dependent sender is a sender type whose completion signatures are knowable independent of an execution environment.

[Editor's note: Change [execution.syn] as follows:]

Header <execution> synopsis [execution.syn]

```
namespace std::execution {  
    ... as before ...  
  
    template<class Sndr, class... Env =env<>>
```

```

    concept sender_in = see below;

    template<class Sndr>
        concept dependent_sender = see below;

    template<class Sndr, class Rcvr>
        concept sender_to = see below;

    template<class... Ts>
        struct type-list;                                //
        exposition only

    // [exec.getcomplsigs], completion signatures
    struct get_completion_signatures_t;
        inline constexpr get_completion_signatures_t
    get_completion_signatures {};

    [ Editor's note: This alias is moved below and modified.
      ]
    template<class Sndr, class... Env>
        requires sender_in<Sndr, Env...>
        using completion_signatures_of_t = call-result-
    t<get_completion_signatures_t, Sndr, Env...>;

    template<class... Ts>
        using decayed-tuple = tuple<decay_t<Ts>...>;    //
        exposition only

    ... as before ...

    template<class Sndr, class... Env>
        using single-sender-value-type = see below;    //
        exposition only

    template<class Sndr, class... Env>
        concept single-sender = see below;            //
        exposition only

    ... as before ...

    // [exec.util], sender and receiver utilities
    // [exec.util.cmplsig] completion signatures
    template<class Fn>
        concept completion-signature = see below;    //
        exposition only

    template<completion-signature... Fns>
        struct completion_signatures {};

    template<class Sigs>
        concept valid-completion-signatures = see below;    //
        exposition only

    struct dependent_sender_error : exception {};

    // [exec.getcomplsigs]
    template<class Sndr, class... Env>
        consteval auto get_completion_signatures() -> valid-completion-
    signatures auto;

```

```

template<class Sndr, class... Env>
    requires sender_in<Sndr, Env...>
        using completion_signatures_of_t =
            decltype(get_completion_signatures<Sndr, Env...>());

// [exec.util.emplsig.trans]
template<
    valid_completion_signatures InputSignatures,
    valid_completion_signatures AdditionalSignatures =
        completion_signatures<>,
    template<class...> class SetValue = see_below,
    template<class> class SetError = see_below,
    valid_completion_signatures SetStopped =
        completion_signatures<set_stopped_t()>>
    using transform_completion_signatures = completion_signatures<see-
        below>;

template<
    sender Sndr,
    class Env = env<>,
    valid_completion_signatures AdditionalSignatures =
        completion_signatures<>,
    template<class...> class SetValue = see_below,
    template<class> class SetError = see_below,
    valid_completion_signatures SetStopped =
        completion_signatures<set_stopped_t()>>
    requires sender_in<Sndr, Env>
    using transform_completion_signatures_of =
        transform_completion_signatures<
            completion_signatures_of_t<Sndr, Env>,
            AdditionalSignatures, SetValue, SetError, SetStopped>;

// [exec.run.loop], run_loop
class run_loop;

... as before ...
}

```

- 1 The exposition-only type `variant-or-empty<Ts...>` is defined as follows ... as before
- 2 For types `Sndr` and `pack of types Env`, let `CS` be `completion_signatures_of_t<Sndr, Env...>`. Then `single-sender-value-type<Sndr, Env...>` is ill-formed if `CS` is ill-formed or if `sizeof...(Env) > 1` is true; otherwise, it is an alias for:
 - (2.1) — `value_types_of_t<Sndr, Env gather_signatures<set_value_t, CS, decay_t, type_identity_t>` if that type is well-formed,
 - (2.2) — Otherwise, void if `value_types_of_t<Sndr, Env gather_signatures<set_value_t, CS, tuple, variant>` is `variant<tuple<>>` or `variant<>`,

- (2.3) — Otherwise, ~~value_types_of_t<Sndr, Env~~ gather-signatures<set_value_t, CS, decayed-tuple, type_identity_t> if that type is well-formed,
- (2.4) — Otherwise, *single-sender-value-type*<Sndr, Env...> is ill-formed.

3 The exposition-only concept *single-sender* is defined as follows:

```
namespace std::execution {
    template<class Sndr, class... Env>
    concept single-sender = sender_in<Sndr, Env...> &&
        requires {
            typename single-sender-value-type<Sndr, Env...>;
        };
}
```

? A type satisfies and models the exposition-only concept *valid-completion-signatures* if it is a specialization of the *completion_signatures* class template.

[Editor's note: Modify [exec.snd.general] as follows:]

- 1 Subclauses [exec.factories] and [exec.adapt] define customizable algorithms that return senders. Each algorithm has a default implementation. Let *sndr* be the result of an invocation of such an algorithm or an object equal to the result ([concepts.equality]), and let *Sndr* be `decltype((sndr))`. Let *rcvr* be a receiver of type *Rcvr* with associated environment *env* of type *Env* such that *sender_to*<*Sndr*, *Rcvr*> is true. For the default implementation of the algorithm that produced *sndr*, connecting *sndr* to *rcvr* and starting the resulting operation state ([exec.async.ops]) necessarily results in the potential evaluation ([basic.def.odr]) of a set of completion operations whose first argument is a subexpression equal to *rcvr*. Let *Sigs* be a pack of completion signatures corresponding to this set of completion operations. ~~Then , and let *CS* be the type of the~~ expression ~~`get_completion_signatures(sndr, env)`~~ `get_completion_signatures<Sndr, Env>()`. Then *CS* is a specialization of the class template *completion_signatures* ([exec.util.cmplsig]), the set of whose template arguments is *Sigs*. If none of the types in *Sigs* are dependent on the type *Env*, then the expression `get_completion_signatures<Sndr>()` is well-formed and its type is *CS*. If a user-provided implementation of the algorithm that produced *sndr* is selected instead of the default: [Editor's note: Reformatted into a list.]
- (1.1) — Any completion signature that is in the set of types denoted by *completion_signatures_of_t*<*Sndr*, *Env*> and that is not part of *Sigs* shall correspond to error or stopped completion operations, unless otherwise specified.
- (1.2) — If none of the types in *Sigs* are dependent on the type *Env*, then *completion_signatures_of_t*<*Sndr*> and *completion_signatures_of_t*<*Sndr*, *Env*> shall denote the same type.

[Editor's note: In [exec.snd.expos], change para 2 as follows:]

- ² For a queryable object *env*, *FWD-ENV*(*env*) is an expression whose type satisfies queryable such that for a query object *q* and a pack of subexpressions *as...*, the expression *FWD-ENV*(*env*).*query*(*q*, *as...*) is ill-formed if *forwarding_query*(*q*) is false; otherwise, it is expression-equivalent to *env.query*(*q*, *as...*). The type *FWD-ENV-T*(*Env*) is *decltype*(*FWD-ENV*(*declval*<*Env*>()).).

[Editor's note: In [exec.snd.expos], insert the following paragraph after para 22 and before para 23 (thus moving the exposition-only concept out of para 24 and into its own para so it can be used from elsewhere):]

- ? Let *valid-specialization* be the following concept:

```
template<template<class...> class T, class... Args>
    concept valid-specialization = requires { typename
        T<Args...>; }; // exposition only
```

[Editor's note: In [exec.snd.expos] para 23 add the mandate shown below:]

```
template<class Tag, class Data = see below, class... Child>
    constexpr auto make-sender(Tag tag, Data&& data, Child&&...
        child);
```

- ²³ *Mandates*: The following expressions are true:

- (23.1) — *semiregular*<Tag>
- (23.2) — *movable-value*<Data>
- (23.3) — (*sender*<Child> &&...)
- (23.4) — *dependent_sender*<Sndr> || *sender_in*<Sndr>, where *Sndr* is *basic-sender*<Tag, Data, Child...> as defined below.

Recommended practice: When this mandate fails because *get_completion_signatures*<Sndr>() would exit with an exception, implementations are encouraged to include information about the exception in the resulting diagnostic.

[Editor's note: In [exec.snd.expos] para 24, change the definition of the exposition-only *basic-sender* template as follows:]

- ²⁴ *Returns*: A prvalue of type *basic-sender*<Tag, decay_t<Data>, decay_t<Child>...> that has been direct-list-initialized with the forwarded arguments, where *basic-sender* is the following exposition-only class template except as noted below.

```
namespace std::execution {
    template<class Tag>
        concept completion-tag = // exposition only
            same_as<Tag, set_value_t> || same_as<Tag, set_error_t>
                || same_as<Tag, set_stopped_t>;

    template<template<class...> class T, class... Args>
```

```

concept valid-specialization = requires { typename
T<Args...>; }; // exposition only

struct default-impls { // exposition only
    static constexpr auto get-attrs = see below;
    static constexpr auto get-env = see below;
    static constexpr auto get-state = see below;
    static constexpr auto start = see below;
    static constexpr auto complete = see below;

    template<class Sndr, class... Env>
    static consteval void check-types().;
};

... as before ...

template <class Sndr>
using data-type = decltype(declval<Sndr>().template
get<1>()); // exposition only

template <class Sndr, size_t I = 0>
    using child-type = decltype(declval<Sndr>().template
        get<I+2>()); // exposition only

... as before ...

template<class Sndr, class... Env>
using completion-signatures-for = see below; //
exposition only

template<class Tag, class Data, class... Child>
struct basic-sender : product-type<Tag, Data, Child...>
{ // exposition only
    using sender_concept = sender_t;
    using indices-for = index_sequence_for<Child...>; //
    exposition only

    decltype(auto) get_env() const noexcept {
        auto& [_ , data, ...child] = *this;
        return impls-for<Tag>::get-attrs(data, child...);
    }

    template<decays-to<basic-sender> Self, receiver Rcvr>
    auto connect(this Self&& self, Rcvr rcvr) noexcept(see
        below)
        -> basic-operation<Self, Rcvr> {
        return {std::forward<Self>(self), std::move(rcvr)};
    }

    template<decays-to<basic-sender> Self, class... Env>
    static constexpr auto get_completion_signatures().;
    {this Self&& self, Env&&... env} noexcept
    -> completion-signatures-for<Self, Env...>{
    return {};
}

};
}

```

[Editor's note: In [exec.snd.expos], replace para 39 with the paragraphs shown below and renumber subsequent paragraphs:]

- 39 Let *Sndr* be a (possibly *const*-qualified) specialization *basic-sender* or an lvalue reference of such, let *Rcvr* be the type of a receiver with an associated environment of type *Env*. If the type *basic-operation*<*Sndr*, *Rcvr*> is well-formed, let *op* be an lvalue subexpression of that type. Then *completion-signatures-for*<*Sndr*, *Env*> denotes a specialization of *completion_signatures*, the set of whose template arguments corresponds to the set of completion operations that are potentially evaluated ([*basic.def.odr*]) as a result of evaluating *op.start()*. Otherwise, *completion-signatures-for*<*Sndr*, *Env*> is ill-formed. If *completion-signatures-for*<*Sndr*, *Env*> is well-formed and its type is not dependent upon the type *Env*, *completion-signatures-for*<*Sndr*> is well-formed and denotes the same type; otherwise, *completion-signatures-for*<*Sndr*> is ill-formed.

```
template <class Sndr, class... Env>
    static consteval void default-impls::check-types();
```

- ? Let *Is* be the pack of integral template arguments of the *integer_sequence* specialization denoted by *indices-for*<*Sndr*>.

- ? *Effects*: Equivalent to:

```
(get_completion_signatures<child-type<Sndr, Is>, FWD-ENV-T(Env)...>(), ...)
```

- ? *Remarks*: For any types *T*, *S*, and pack *E*, let *e* be the expression *impls-for*<*T*>::*check-types*<*S*, *E*...>(). Then exactly one of the following is true:

- (?.1) — *e* is ill-formed, or
- (?.3) — The evaluation of *e* exits with an exception, or
- (?.2) — *e* is a core constant expression.

When *e* is a core constant expression, the types *S*, *E*... uniquely determine a set of completion signatures.

```
template<class Tag, class Data, class... Child>
    template <class Sndr, class... Env>
        constexpr auto basic-sender<Tag, Data,
            Child...>::get_completion_signatures();
```

- ? Let *Rcvr* be the type of a receiver whose environment has type *E*, where *E* is the first type in the list *Env*..., *env*<>. Let *CHECK-TYPES*() be the expression *impls-for*<*Tag*>::template *check-types*<*Sndr*, *E*>(), and let *CS* be a type determined as follows:

- (?.1) — If *CHECK-TYPES*() is a core constant expression, let *op* be an lvalue subexpression whose type is *connect_result_t*<*Sndr*, *Rcvr*>. Then *CS* is the specialization of *completion_signatures* the set of whose template

arguments correspond to the set of completion operations that are potentially evaluated ([basic.def.odr]) as a result of evaluating `op.start()`.

(?.2) — Otherwise, CS is `completion_signatures<>`.

? *Constraints:* `CHECK-TYPES()` is a well-formed expression.

? *Effects:* Equivalent to

```
CHECK-TYPES();
return CS();
```

[Editor's note: Add the following new paragraphs to the end of [exec.snd.expos]]

```
? template<class... Fns>
struct overload-set : Fns... {
    using Fns::operator()...;
};
```

[Editor's note: The following is moved from [exec.on] para 6 and modified.]

```
? struct not-a-sender {
    using sender_concept = sender_t;

    template<class Sndr>
        static constexpr auto get_completion_signatures() ->
            completion_signatures<> {
            throw unspecified;
        }
};
```

```
? constexpr void decay-copyable-result-datums(auto cs) {
    cs.for-each([]<class Tag, class... Ts>(Tag*)(Ts...)) {
        if constexpr (!is_constructible_v<decay_t<Ts>, Ts> &&...))
            throw unspecified;
    });
}
```

[Editor's note: Change [exec.snd.concepts] para 1 and add a new para after 1 as follows:]

1 The sender concept ... *as before* ... to produce an operation state.

```
namespace std::execution {
    template<class Sigs>
    concept valid-completion-signatures = see below;
    // exposition only

    template<auto>
    concept is-constant = true;
    // exposition only

    template<class Sndr>
        concept is-sender =
            // exposition only
            derived_from<typename Sndr::sender_concept,
            sender_t>;

    template<class Sndr>
```

```

concept enable_sender =
    // exposition only
    is_sender<Sndr> ||
        is_awaitable<Sndr, env_promise<env<>>>>;
    // [exec.awaitable]

template<class Sndr>
constexpr bool is_dependent_sender_helper() try {
    // exposition only
    get_completion_signatures<Sndr>().;
    return false;
} catch (dependent_sender_error&) {
    return true;
}

template<class Sndr>
concept sender =
    bool(enable_sender<remove_cvref_t<Sndr>>) &&
    requires (const remove_cvref_t<Sndr>& sndr) {
        { get_env(sndr) } -> queryable;
    } &&
    move_constructible<remove_cvref_t<Sndr>> &&
    constructible_from<remove_cvref_t<Sndr>, Sndr>;

template<class Sndr, class... Env = env<>>
concept sender_in =
    sender<Sndr> &&
    (sizeof...(Env) <= 1) &&
    (queryable<Env> &&...) &&
    is_constant<get_completion_signatures<Sndr, Env...>
        ()>;
requires (Sndr&& sndr, Env&& env) {
    { get_completion_signatures(std::forward<Sndr>
    (sndr), std::forward<Env>(env)) }
-> valid_completion_signatures;
};

template<class Sndr>
concept dependent_sender =
    sender<Sndr> && bool constant<is_dependent_sender_helper<Sndr>().>::value;

template<class Sndr, class Rcvr>
concept sender_to =
    sender_in<Sndr, env_of_t<Rcvr>> &&
    receiver_of<Rcvr, completion_signatures_of_t<Sndr,
    env_of_t<Rcvr>>> &&
    requires (Sndr&& sndr, Rcvr&& rcvr) {
        connect(std::forward<Sndr>(sndr),
        std::forward<Rcvr>(rcvr));
    };
}

```

- ? — For a type Sndr, if sender<Sndr> is true and dependent_sender<Sndr> is false, then Sndr is a non-dependent sender ([exec.async.ops]).

[Editor's note: Strike [exec.snd.concepts] para 3 (this para is moved to [execution.syn]):]

- 3 ~~A type models the exposition-only concept *valid-completion-signatures* if it denotes a specialization of the *completion_signatures* class template.~~

[Editor's note: Change [exec.snd.concepts] para 4 as follows (so that the exposition-only *sender-of* concept tests for sender-ness with no environment as opposed to the empty environment, *env<>*):]

- 4 The exposition-only concepts *sender-of* and *sender-in-of* define the requirements for a sender type that completes with a given unique set of value result types.

```
namespace std::execution {
    template<class... As>
        using value-signature = set_value_t(As...);           //
        exposition only

    template<class Sndr, class Env, class... Values>
        concept sender-in-of =
            sender_in<Sndr, Env> &&
            MATCHING-SIG(_____, // see
            [exec.general]
            set_value_t(Values...),
            value_types_of_t<Sndr, Env, value-signature,
            type_identity_t>);

    template<class Sndr, class... Values>
        concept sender-of = sender-in-of<Sndr, env<>,
        Values...>;

    template<class Sndr, class SetValue, class... Env>
        concept sender-in-of-impl = _____ // exposition only
        sender_in<Sndr, Env...> &&
        MATCHING-SIG(SetValue, _____ //
        see [exec.general]
        gather-signatures<set_value_t, _____ //
        see [exec.util.cmplsig].

        completion_signatures_of_t<Sndr, Env...>,
        value-signature,
        type_identity_t>);

    template<class Sndr, class Env, class... Values>
        concept sender-in-of = _____ // exposition only
        sender-in-of-impl<Sndr, set_value_t(Values...),
        Env>;

    template<class Sndr, class... Values>
        concept sender-of = _____ // exposition only
        sender-in-of-impl<Sndr, set_value_t(Values...)>;
}
```

[Editor's note: Change [exec.awaitable] p 1-4 as follows:]

- 1 The sender concepts recognize awaitables as senders. For [exec], an *awaitable* is an expression that would be well-formed as the operand of a *co_await* expression within a given context.

- 2 For a subexpression *c*, let `GET-AWAITER(c, p)` be expression-equivalent to the series of transformations and conversions applied to *c* as the operand of an *await-expression* in a coroutine, resulting in lvalue *e* as described by `[expr.await]`, where *p* is an lvalue referring to the coroutine's promise, which has type `Promise`.

[Note 1: This includes the invocation of the promise type's `await_transform` member if any, the invocation of the operator `co_await` picked by overload resolution if any, and any necessary implicit conversions and materializations. — end note]

Let `GET-AWAITER(c)` be expression-equivalent to `GET-AWAITER(c, q)` where *q* is an lvalue of an unspecified empty class type *none-such* that lacks an `await_transform` member, and where `coroutine_handle<none-such>` behaves as `coroutine_handle<void>`.

- 3 Let *is-awaitable* be the following exposition-only concept: [Editor's note: NB: there are added ellipses in the following code block.]

```
template<class T>
    concept await-suspend-result = see below;

template<class A, class... Promise>
    concept is-awaiter = // exposition only
        requires (A& a, coroutine_handle<Promise...> h) {
            a.await_ready() ? 1 : 0;
            { a.await_suspend(h) } -> await-suspend-result;
            a.await_resume();
        };

template<class C, class... Promise>
    concept is-awaitable =
        requires (C (*fc)() noexcept, Promise&... p) {
            { GET-AWAITER(fc(), p...) } -> is-
                awriter<Promise...>;
        };
};
```

await-suspend-result<*T*> is true if and only if one of the following is true:

- (3.1) — *T* is void, or
- (3.2) — *T* is bool, or
- (3.3) — *T* is a specialization of `coroutine_handle`.

- 4 For a subexpression *c* such that `decltype(c)` is type *C*, and an lvalue *p* of type `Promise`, *await-result-type*<*C*, `Promise`> denotes the type `decltype(GET-AWAITER(c, p).await_resume())` and *await-result-type*<*C*> denotes the type `decltype(GET-AWAITER(c).await_resume())`.

[Editor's note: Change `[exec.getcompsigs]` as follows:]

- 1 `get_completion_signatures` is a customization point object. Let *sndr* be an expression such that `decltype(sndr)` is *Sndr*, and let *env* be an expression such that `decltype(env)` is *Env*. Let *new_sndr* be the expression

`transform_sender(decltype(get-domain-late(sndr, env)){}, sndr, env)`, and let `NewSndr` be `decltype((new_sndr))`. Then `get_completion_signatures(sndr, env)` is expression-equivalent to `(void(sndr), void(env), CS())` except that `void(sndr)` and `void(env)` are indeterminately sequenced, where `CS` is:

- (1.1) — `decltype(new_sndr.get_completion_signatures(env))` if that type is well-formed,
- (1.2) — Otherwise, `remove_cvref_t<NewSndr>::completion_signatures` if that type is well-formed,
- (1.3) — Otherwise, if `is-awaitable<NewSndr, env-promise<Env>>` is true, then:

```
completion_signatures<
    SET-VALUE-SIG(await-result-type<NewSndr, env-
        promise<Env>>), // ([exec.snd.concepts])
    set_error_t(exception_ptr),
    set_stopped_t()>
```

- (1.4) — Otherwise, `CS` is ill-formed.

```
template <class Sndr, class... Env>
constexpr auto get_completion_signatures() -> valid-completion-
    signatures auto;
```

- ? Let `except` be an rvalue subexpression of an unspecified class type `Except` such that `move_constructible<Except> && derived_from<Except, exception>` is true. Let `CHECKED-COMPLSIGS(e)` be `e` if `e` is a core constant expression whose type satisfies `valid-completion-signatures`; otherwise, it is the following expression:

```
(e, throw except, completion_signatures())
```

Let `get-complsig<Sndr, Env...>()` be expression-equivalent to `remove_reference_t<Sndr>::template get_completion_signatures<Sndr, Env...>()`, and let `NewSndr` be `Sndr` if `sizeof...(Env) == 0` is true; otherwise, `decltype(s)` where `s` is the following expression:

```
transform_sender(
    get-domain-late(declval<Sndr>(), declval<Env>()...),
    declval<Sndr>(),
    declval<Env>()...)
```

- ? *Constraints:* `sizeof...(Env) <= 1` is true.
- ? *Effects:* Equivalent to: return `e`; where `e` is expression-equivalent to the following:
 - (?.1) — `CHECKED-COMPLSIGS(get-complsig<NewSndr, Env...>())` if `get-complsig<NewSndr, Env...>()` is a well-formed expression.
 - (?.2) — Otherwise, `CHECKED-COMPLSIGS(get-complsig<NewSndr>())` if `get-complsig<NewSndr>()` is a well-formed expression.

(?.3) — Otherwise,

```
completion_signatures<
    SET-VALUE-SIG(await-result-type<NewSndr, env-
        promise<Env>...>), // ([exec.snd.concepts])
    set_error_t(exception_ptr),
    set_stopped_t()>
```

if *is-awaitable*<NewSndr, env-promise<Env>...> is true.

(?.4) — Otherwise, (throw *dependent-sender-error*(), completion_signatures()) if sizeof...(Env) == 0 is true, where *dependent-sender-error* is *dependent_sender_error* or an unspecified type derived publicly and unambiguously from *dependent_sender_error*.

(?.5) — Otherwise, (throw *except*, completion_signatures()).

? Given a type Env, if completion_signatures_of_t<Sndr> and completion_signatures_of_t<Sndr, Env> are both well-formed, they shall denote the same type.

2 Let rcvr be an rvalue whose type [Editor's note: ... as before].

[Editor's note: At the very bottom of [exec.connect], change the *Mandates* of para 6 as follows:]

6 The expression connect(sndr, rcvr) is expression-equivalent to:

(6.1) — new_sndr.connect(rcvr) if that expression is well-formed.

Mandates: The type of the expression above satisfies operation_state.

(6.2) — Otherwise, *connect-awaitable*(new_sndr, rcvr).

Mandates: ~~sender<Sndr> && receiver<Rcvr>~~ The following are true:

(6.3) — sender_in<Sndr, env_of_t<Rcvr>>

(6.4) — receiver_of<Rcvr, _____ completion_signatures_of_t<Sndr, _____ env_of_t<Rcvr>>>

[Editor's note: In [exec.read.env] para 3, make the following change:]

3 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for read_env as follows:

```
namespace std::execution {
    template<>
    struct impls-for<decayed-typeof<read_env>> : default-impls {
        static constexpr auto start =
            [](auto query, auto& rcvr) noexcept -> void {
                TRY-SET-VALUE(std::move(rcvr),
                    query(get_env(rcvr)));
            };
    };
}
```

```

    template<class Sndr, class Env>
    static consteval void check-types();
};
}

```

```

template<class Sndr, class Env>
static consteval void check-types();

```

- (3.1) — Let `Q` be `decay_t<data-type<Sndr>>`.
- (3.2) — *Throws*: An exception of an unspecified type derived from `exception` if the expression `Q()(env)` is ill-formed or has type `void`, where `env` is an lvalue subexpression whose type is `Env`.

[Editor's note: Change [exec.adapt.general] as follows:]

- (3.4) — When a parent sender is connected to a receiver `rcvr`, any receiver used to connect a child sender has an associated environment equal to `FWD-ENV(get_env(rcvr))`.
- (3.?) — An adaptor whose child senders are all non-dependent ([async.ops]) is itself non-dependent.
- (3.5) — These requirements apply to any function that is selected by the implementation of the sender adaptor.
- (3.?) — Recommended practice: Implementors are encouraged to use the completion signatures of the adaptors to communicate type errors to users and to propagate any such type errors from child senders.

[Editor's note: Change [exec.write.env] as follows (see [P3284R4] for [exec.write.env], to be voted on in Bulgaria):]

- 1 `write_env` is a sender adaptor that accepts a sender and a queryable object, and that returns a sender that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the queryable argument `env` to the result of `get_env(rcvr)`.
- 2 `write_env` is a customization point object. For some subexpressions `sndr` and `env`, if `decltype((sndr))` does not satisfy *sender* or if `decltype((env))` does not satisfy *queryable*, the expression `write_env(sndr, env)` is ill-formed. Otherwise, it is expression-equivalent to `make_sender(write_env, env, sndr)`.
- 3 Let *write-env-t* denote the type `decltype(auto(write_env))`. The exposition-only class template *impls-for* ([exec.snd.expos]) is specialized for *write-env-t* as follows:

```

template<>
struct impls-for<write-env-t> : default-impls {
    static constexpr auto join-env(const auto& state, const
        auto& env) noexcept {
        return see below;
    }
};

```

```

}

static constexpr auto get-env =
    [](auto, const auto& state, const auto& rcvr)
    noexcept {
        return see below join-env(state, FWD-ENV(get_env(rcvr)).);
    };

template<class Sndr, class... Env>
static constexpr void check-types().;
};

```

Invocation of `impls-for<write-env-t>::get-envjoin-env` returns an object `e` such that

- (3.1) — `decltype(e)` models *queryable* and
- (3.2) — given a query object `q`, the expression `e.query(q)` is expression-equivalent to `state.query(q)` if that expression is valid; otherwise, `e.query(q)` is expression-equivalent to `FWD-ENV(get_env(rcvr))env.query(q)`.
- (3.3) — For type `Sndr` and pack of types `Env`, let `State` be `data-type<Sndr>` and let `JoinEnv` be the pack `decltype(join-env(declval<State>(), FWD-ENV(declval<Env>())))`. Then `impls-for<write-env-t>::check-types<Sndr, Env...>()` is expression-equivalent to `get_completion_signatures<child-type<Sndr>, JoinEnv...>()`.

[Editor's note: Change [exec.schedule.from] para 4 and insert a new para between 6 and 7 as follows:]

- 4 The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `schedule_from_t` as follows:

```

namespace std::execution {
    template<>
    struct impls-for<schedule_from_t> : default-impls {
        static constexpr auto get-attrs = see below;
        static constexpr auto get-state = see below;
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
        static constexpr void check-types().;
    };
}

```

- 5 The member `... as before ...`
- 6 The member `impls-for<schedule_from_t>::get-state` is initialized with a callable object equivalent to the following lambda: [Editor's note: This integrates the resolution from LWG#4203.]

```
[ ]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
    noexcept(see below)
    requires sender_in<child-type<Sndr>, FWD-ENV-
T(env_of_t<Rcvr>)> {

    auto& [_ , sch, child] = sndr;
    ... as before ...
```

```
template<class Sndr, class... Env>
static consteval void check-types();
```

? *Effects*: Equivalent to:

```
get_completion_signatures<schedule_result_t<data-
    type<Sndr>>, FWD-ENV-T(Env)...>();
auto cs = get_completion_signatures<child-type<Sndr>, FWD-
ENV-T(Env)...>();
decay-copyable-result-datums(cs); // see [exec.snd.expos]
```

7 Objects of the local class *state-type* ... as before ...

8 Let Sigs be a pack of the arguments to the *completion_signatures* specialization named by *completion_signatures_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>*. Let *as-tuple* be an alias template such that *as-tuple<Tag(Args...)>* denotes the type *decayed-tuple<Tag, Args...>*. Then *variant_t* denotes the type *variant<monostate, as-tuple<Sigs>...>*, except with duplicate types removed.

[Editor's note: Change [exec.on] para 6 as follows (*not-a-sender* is moved to [exec.snd.expos]):]

6 Otherwise: Let *not-a-scheduler* be an unspecified empty class type, ~~and let *not-a-sender* be the exposition-only type:~~

```
struct not-a-sender {
    using sender_concept = sender_t;

    auto get_completion_signatures(auto&&) const {
        return see below;
    }
};
```

~~where the member function *get_completion_signatures* returns an object of a type that is not a specialization of the *completion_signatures* class template.~~

[Editor's note: Delete [exec.on] para 9 as follows:]

9 ~~*Recommended practice*: Implementations should use the return type of *not-a-sender::get_completion_signatures* to inform users that their usage of *on* is incorrect because there is no available scheduler onto which to restore execution.~~

[Editor's note: Change [exec.then] para 4 as follows:]

4 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *then-cpo* as follows:

```

namespace std::execution {
    template<>
    struct impls-for<decayed-typeof<then-cpo>> : default-
        impls {
        static constexpr auto complete =
            []<class Tag, class... Args>
            (auto, auto& fn, auto& rcvr, Tag, Args&&... args)
            noexcept -> void {
                if constexpr (same_as<Tag, decayed-typeof<set-
                    cpo>>) {
                    TRY-SET-VALUE(rcvr,
                        invoke(std::move(fn),
                            std::forward<Args>(args)...));
                } else {
                    Tag()(std::move(rcvr), std::forward<Args>
                        (args)...);
                }
            };

        template<class Sndr, class... Env>
        static consteval void check-types();
    };
}

```

? template<class Sndr, class... Env>
static consteval void check-types();

(?.1) — *Effects*: Equivalent to:

```

auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-
    T(Env)...>();
auto fn = []<class... Ts>(set_value_t(*) (Ts...)) {
    if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>,
        Ts...>)
        throw unspecified;
};
cs.for-each(overload-set{fn, [] (auto){}});

```

[Editor's note: Change [exec.let] paras 5 and 6 and insert a new para after 6 as follows:]

- 5 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *let-cpo* as follows:

```

namespace std::execution {
    template<class State, class Rcvr, class... Args>
    void let-bind(State& state, Rcvr& rcvr, Args&&... args);
    // exposition only

    template<>
    struct impls-for<decayed-typeof<let-cpo>> : default-
        impls {
        static constexpr auto get-state = see below;
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
        static consteval void check-types();
    };
}

```

6 Let *receiver2* denote the following exposition-only class template:

```
namespace std::execution {  
    ... as before ...  
}
```

Invocation of the function *receiver2::get_env* returns an object *e* such that

- (6.1) — `decltype(e)` models *queryable* and
- (6.2) — given a query object *q*, the expression *e*.*query*(*q*) is expression-equivalent to *env*.*query*(*q*) if that expression is valid; otherwise, if the type of *q* satisfies forwarding-query, *e*.*query*(*q*) is expression-equivalent to *get_env(rcvr)*.*query*(*q*); otherwise, *e*.*query*(*q*) is ill-formed.

```
? template<class Sndr, class... Env>  
consteval void check-types();
```

- (?.1) — *Effects*: Equivalent to:

```
using LetFn = remove_cvref_t<data-type<Sndr>>;  
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-  
    T(Env)...>();  
auto fn = []<class... Ts>(decayed-typeof<set-cpo>(*) (Ts...)) {  
    if constexpr (!is-valid-let-sender)  
        throw unspecified;  
};  
cs.for-each(overload-set(fn, [] (auto){}));
```

where *is-valid-let-sender* is true if and only if all of the following are true:

- (?.1.1) — `(constructible_from<decay_t<Ts>, Ts> &&...)`
- (?.1.2) — `invocable<LetFn, decay_t<Ts>&...>`
- (?.1.3) — `sender<invoke_result_t<LetFn, decay_t<Ts>&...>>`
- (?.1.4) — `sizeof...(Env) == 0 || sender_in<invoke_result_t<LetFn, decay_t<Ts>&...>, env-t...>`

where *env-t* is the pack `decltype(let-cpo.transform_env(declval<Sndr>(), declval<Env>()))`.

[Editor's note: The following changes are the proposed resolutions to [cplusplus/sender-receiver#316](#) and [cplusplus/sender-receiver#318](#).]

7 *impls-for<decayed-typeof<let-cpo>>::get-state* is initialized with a callable object equivalent to the following:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)  
    requires see below {  
    auto& [_, fn, child] = sndr;  
    using fn_t = decay_t<decltype(fn)>;  
    using env_t = decltype(let-env(child));  
    using args_variant_t = see below;  
    using ops2_variant_t = see below;
```

```
... as before ...
}
```

- 8 Let Sigs be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<child-type<Sndr>, FWD-ENV-T(env_of_t<Rcvr>)>`. Let LetSigs be a pack of those types in Sigs with a return type of `decayed-typeof<set-cpo>`. Let *as-tuple* be an alias template such that *as-tuple*<Tag(Args...)> denotes the type *decayed-tuple*<Args...>. Then `args_variant_t` denotes the type `variant<monostate, as-tuple<LetSigs>...>` except with duplicate types removed.

- 9 Given a type Tag and a pack Args, let *as-sndr2* be an alias template such that *as-sndr2*<Tag(Args...)> denotes the type *call-result-t*<Fn, decay_t<Args>&...>. Then `ops2_variant_t` denotes the type

```
variant<monostate, connect_result_t<as-sndr2<LetSigs>,
receiver2<Rcvr, Envenv_t>>...>
```

except with duplicate types removed.

- 10 The *requires-clause* constraining the above lambda is satisfied if and only if the types `args_variant_t` and `ops2_variant_t` are well-formed.

- 11 The exposition-only function template *let-bind* has effects equivalent to: *... as before ...*

- 12 *... as before ...*

[Editor's note: The following change to [exec.let] para 13 is the proposed resolution to cplusplus/sender-receiver#319.]

- 13 Let *sndr* and *env* be subexpressions, and let *Sndr* be `decltype((sndr))`. If *sender-for*<*Sndr*, *decayed-typeof*<*let-cpo*>> is false, then the expression *let-cpo*.transform_env(*sndr*, *env*) is ill-formed. Otherwise, it is equal to ~~*JOIN-ENV(let-env(sndr), FWD-ENV(env))*~~:

```
auto& [_, __, child] = sndr;
return JOIN-ENV(let-env(child), FWD-ENV(env)).;
```

[Editor's note: Change [exec.bulk] para 3 and insert a new para after 5 as follows:]

- 3 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for `bulk_t` as follows:

```
namespace std::execution {
    template<>
    struct impls-for<bulk_t> : default-impls {
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
        static constexpr void check-types().;
    };
}
```


4 The member *impls-for*<bulk_t>::complete is ... as before ...

5 ... as before ...

? `template<class Sndr, class... Env>
consteval void check-types();`

(?.1) — *Effects*: Equivalent to:

```
auto cs = get_completion_signatures<child-type<Sndr>, FWD-ENV-  
T(Env)...>();  
auto fn = []<class... Ts>(set_value_t*)(Ts...) {  
    if constexpr (!invocable<remove_cvref_t<data-type<Sndr>>,  
        Ts&...>)  
        throw unspecified;  
};  
cs.for-each(overload-set{fn, [](auto){}});
```

6 Let the subexpression *out_sndr* denote ... as before ...

[Editor's note: Change [exec.split] para 3 and insert a new para after 3 as follows:]

3 The name *split* denotes a pipeable sender adaptor object. ~~For a subexpression *sndr*, let *Sndr* be `decltype((sndr))`. If `sender_in<Sndr, split-env>` is false, `split(sndr)` is ill-formed.~~

? The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *split_t* as follows:

```
namespace std::execution {  
    template<>  
    struct impls-for<split_t> : default-impls {  
        template<class Sndr>  
        static consteval void check-types() {  
            auto cs = get_completion_signatures<child-  
                type<Sndr>, split-env>();  
            decay-copyable-result-datums(cs); // see  
                [exec.snd.expos]  
        }  
    };  
}
```

[Editor's note: Change [exec.when.all] paras 2-9 and insert two new paras after 4 as follows:]

2 The names *when_all* and *when_all_with_variant* denote customization point objects. Let *sndrs* be a pack of subexpressions, let *Sndrs* be a pack of the types `decltype((sndrs))...`, and let *CD* be the type `common_type_t<decltype(get-domain-early(sndrs))...>`, and let *CD2* be *CD* if *CD* is well-formed, and *default_domain* otherwise. The expressions *when_all*(*sndrs*...) and *when_all_with_variant*(*sndrs*...) are ill-formed if any of the following is true:

(2.1) — `sizeof...(sndrs)` is 0, or

(2.2) — `(sender<Sndrs> && ...)` is false, ~~or~~

(2.3) — ~~*CD* is ill-formed.~~

- 3 The expression `when_all(sndrs...)` is expression-equivalent to:

```
transform_sender(CD(+)CD2(), make_sender(when_all, {}),  
sndrs...))
```

- 4 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for `when_all_t` as follows:

```
namespace std::execution {  
    template<>  
    struct impls-for<when_all_t> : default-impls {  
        static constexpr auto get-attrs = see below;  
        static constexpr auto get-env = see below;  
        static constexpr auto get-state = see below;  
        static constexpr auto start = see below;  
        static constexpr auto complete = see below;  
  
        template<class Sndr, class... Env>  
        static constexpr void check-types();  
    };  
}
```

- ? Let *make-when-all-env* be the following exposition-only function template:

```
template<class Env>  
constexpr auto make-when-all-env(inplace_stop_source&  
    stop_src, Env&& env) noexcept {  
    return see below;  
}
```

Returns an object *e* such that [Editor's note: The following itemized list has been moved here from para 6 and modified as indicated.]

- (?.1) — `decltype(e)` models queryable, and
- (?.2) — `e.query(get_stop_token)` is expression-equivalent to `stop_src.get_token()`, and
- (?.3) — given a query object *q* with type other than `cv stop_token_t` and whose type satisfies *forwarding-query*, `e.query(q)` is expression-equivalent to `env.query(q)`.

Let *when-all-env* be an alias template such that *when-all-env*<Env> denotes the type `decltype(make-when-all-env(declval<inplace_stop_source>(), __declval<Env>()))`.

- ? `template<class Sndr, class... Env>`
`constexpr void check-types();`

- (?.1) — Let *Is* be the pack of integral template arguments of the integer_sequence specialization denoted by *indices-for*<Sndr>.
- (?.2) — Effects: Equivalent to:

```

auto fn = []<class Child>() {
    auto cs = get_completion_signatures<Child, when-all-
        env<Env>...>();
    if constexpr (cs.count-of(set_value) >= 2)
        throw unspecified;
    decay-copyable-result-datums(cs); // see [exec.snd.expos]
};
(fn.template operator()<child-type<Sndr, Is>>(), ...);

```

- (?.3) — Throws: Any exception thrown as a result of evaluating the *Effects*, or an exception of an unspecified type derived from exception when CD is ill-formed.

5 The member *impls-for*<when_all_t>::*get-attrs* ... as before ...

6 The member *impls-for*<when_all_t>::*get-env* is initialized with a callable object equivalent to the following lambda expression:

```

[]<class State, class Rcvr>(auto&& state, const
    Receiver& rcvr) noexcept {
    return see below make-when-all-env(state.stop-src,
        get_env(rcvr));
}

```

~~Returns an object e such that~~

- (6.1) — ~~decltype(e) models queryable, and~~
- (6.2) — ~~e.query(get_stop_token) is expression equivalent to state.stop-src.get_token(), and~~
- (6.3) — ~~given a query object q with type other than cv-stop_token_t, e.query(q) is expression equivalent to get_env(rcvr).query(q).~~

7 The member *impls-for*<when_all_t>::*get-state* is initialized with a callable object equivalent to the following lambda expression:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
    noexcept(e) -> decltype(e) {
    return e;
}

```

where e is the expression

```
std::forward<Sndr>(sndr).apply(make-state<Rcvr>())
```

and where *make-state* is the following exposition-only class template:

```

template<class Sndr, class Env>
concept max-1-sender-in = sender_in<Sndr, Env> &&
    // exposition only@
    {tuple_size_v<value_types_of_t<Sndr, Env, tuple, tuple>>
        <= 1};

enum class disposition { started, error, stopped };
    // exposition only

```

```

template<class Rcvr>
struct make-state {
    template<max-1-sender-in<env_of_t<Rcvr>>>class... Sndrs>
    auto operator()(auto, auto, Sndrs&&... sndrs) const {
        using values_tuple = see below;
        using errors_variant = see below;
        using stop_callback =
            stop_callback_for_t<stop_token_of_t<env_of_t<Rcvr>>,
            on-stop-request>;
... as before ...

```

8 Let *copy-fail* be ... *as before* ...

9 The alias `values_tuple` denotes the type

```

tuple<value_types_of_t<Sndrs, FWD-ENV-T(env_of_t<Rcvr>.),
    decayed-tuple, optional>...>

```

if that type is well-formed; otherwise, `tuple<>`.

[Editor's note: Change [exec.when.all] para 14 as follows:]

14 The expression `when_all_with_variant(sndrs...)` is expression-equivalent to:

```

transform_sender(CD1CD2(), make-
    sender(when_all_with_variant, {}, sndrs...));

```

[Editor's note: Change [exec.into.variant] paras 4-5 as follows (with the change to para 5 being a drive-by fix):]

4 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *into_variant* as follows:

```

namespace std::execution {
    template<>
    struct impls-for<into_variant_t> : default-impls {
        static constexpr auto get-state = see below;
        static constexpr auto complete = see below;

        template<class Sndr, class... Env>
        static consteval void check-types() {
            auto cs = get_completion_signatures<child-
                type<Sndr>, FWD-ENV-T(Env)...>();
            decay-copyable-result-datums(cs); // see
            [exec.snd.expos]
        }
    };
}

```

5 The member `impls-for<into_variant_t>::get-state` is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
noexcept
-> type_identity<value_types_of_t<child-type<Sndr>, FWD-
    ENV-T(env_of_t<Rcvr>.)>> {

```

```
return {};
```

[Editor's note: Change [exec.stopped.opt] as follows. Note: this includes the proposed resolution to cplusplus/sender-receiver#311]

- 2 The name `stopped_as_optional` denotes a pipeable sender adaptor object. For a subexpression `snr`, let `Snr` be `decltype((snr))`. The expression `stopped_as_optional(snr)` is expression-equivalent to:

```
transform_sender(get-domain-early(snr), make-  
sender(stopped_as_optional, {}, snr))
```

except that `snr` is only evaluated once.

- 3 The exposition-only class template `impls-for` ([exec.snd.general]) is specialized for `stopped_as_optional_t` as follows:

```
template<>  
struct impls-for<stopped_as_optional_t> : default-impls {  
    template<class Snr, class... Env>  
    static constexpr void check-types() {  
        default-impls::check-types<Snr, Env...>();  
        if constexpr (!requires {  
            requires (!same_as<void, single-sender-value-  
type<child-type<Snr>, FWD-ENV-T(Env)...>>); })  
            throw unspecified;  
        }  
    }  
};
```

- 3 Let `snr` and `env` be subexpressions such that `Snr` is `decltype((snr))` and `Env` is `decltype((env))`. If `sender-for<Snr, stopped_as_optional_t>` is false; ~~or if the type `single-sender-value-type<Snr, Env>` is ill-formed or void;~~ then the expression `stopped_as_optional.transform_sender(snr, env)` is ill-formed; otherwise, if `sender_in<child-type<Snr>, FWD-ENV-T(Env)>` is false, the expression `stopped_as_optional.transform_sender(snr, env)` is equivalent to `not-a-sender()`; otherwise, it is equivalent to:

```
auto&& [_, _, child] = snr;  
using V = single-sender-value-type<child-type<Snr>, FWD-ENV-  
T(Env)>;  
return let_stopped(  
    then(std::forward_like<Snr>(child),  
        [<class... Ts>(Ts&&... ts)  
        noexcept(is_nothrow_constructible_v<V, Ts...>) {  
            return optional<V>(in_place, std::forward<Ts>(ts)...);  
        }  
    ),  
    [<()>() noexcept { return just(optional<V>()); }]);
```

[Editor's note: Change [exec.sync.wait] as follows:]

- 4 The name `this_thread::sync_wait` denotes a customization point object. For a subexpression `snr`, let `Snr` be `decltype((snr))`. ~~If `sender_in<Snr, sync_wait-
env>` is false, the expression `this_thread::sync_wait(snr)` is ill-~~

~~formed. Otherwise, it~~ The expression `this_thread::sync_wait(sndr)` is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr), sync_wait, sndr)
```

Mandates:

- (4.?) — `sender_in<Sndr, sync-wait-env>` is true.
- (4.1) — The type `sync-wait-result-type<Sndr>` is well-formed.
- (4.2) — `same_as<decltype(e), sync-wait-result-type<Sndr>>` is true, where `e` is the `apply_sender` expression above.

...as before

[Editor's note: Change [exec.sync.wait.var] as follows:]

- 1 The name `this_thread::sync_wait_with_variant` denotes a customization point object. For a subexpression `sndr`, let `Sndr` be `decltype(into_variant(sndr))`. ~~If `sender_in<Sndr, sync-wait-env>` is false, the expression `this_thread::sync_wait(sndr)` is ill-formed. Otherwise, it~~ The expression `this_thread::sync_wait_with_variant(sndr)` is expression-equivalent to the following, except that `sndr` is evaluated only once:

```
apply_sender(get-domain-early(sndr),  
            sync_wait_with_variant, sndr)
```

Mandates:

- (1.?) — `sender_in<Sndr, sync-wait-env>` is true.
- (1.1) — The type `sync-wait-with-variant-result-type<Sndr>` is well-formed.
- (1.2) — `same_as<decltype(e), sync-wait-with-variant-result-type<Sndr>>` is true, where `e` is the `apply_sender` expression above.

- 2 ~~If `callable<sync_wait_t, Sndr>` is false, the expression `sync_wait_with_variant.apply_sender(sndr)` is ill-formed. Otherwise, it~~ The expression `sync_wait_with_variant.apply_sender(sndr)` is equivalent to *...as before*

[Editor's note: Change [exec.util.cmplsig] para 8 and add a new para after 8 as follows:]

```
8 namespace std::execution {  
  template<completion-signature... Fns>  
  struct completion_signatures {  
    template<class Tag>  
    static constexpr size_t count-of(Tag) { return see below; }  
  
    template<class Fn>  
    static constexpr void for-each(Fn&& fn) { // exposition only  
      (std::forward<Fn>(fn)(static_cast<Fns*>(nullptr)), ...);  
    }  
  }  
}
```

```
};

... as before ...
}
```

? For a subexpression tag, let Tag be the decayed type of tag. completion_signatures<Fns...>::count-of(tag) returns the count of function types in Fns... that are of the form Tag(Ts...) where Ts is a pack of types.

[Editor's note: Remove subclause [exec.util.cmplsig.trans].]

[Editor's note: Change [exec.run.loop.types] para 5 as follows:]

5 *run-loop-sender* is an exposition-only type that satisfies sender. ~~For any type~~
~~Env;~~ completion_signatures_of_t<run-loop-sender,~~Env~~> is
 completion_signatures<set_value_t(), set_error_t(exception_ptr),
 set_stopped_t(>).

§ 8 Acknowledgements

I would like to thank Hana Dusíková for her work making constexpr exceptions a reality for C++26. Thanks are also due to David Sankel for his encouragement to investigate using constexpr exceptions as an alternative to TMP hackery, and for giving feedback on an early draft of this paper.

§ 9 References

- [P3068R6] Hana Dusíková. 2024-11-19. Allowing exception throwing in constant-evaluation.
<https://wg21.link/p3068r6>
- [P3164R4] Eric Niebler. Early Diagnostics for Sender Expressions.
<https://isocpp.org/files/papers/P3164R4.html>
- [P3284R4] Eric Niebler. write_env and unstoppable Sender Adaptors.
<https://isocpp.org/files/papers/P3284R4.html>

-
1. <https://godbolt.org/z/rPEqWz693>↵
 2. <https://gist.github.com/ericniebler/0896776ab1c8f5b7f77d7094c0400df5>↵